P420 Working Notes CPMS04

Britt Anderson Britt Anderson

Winter 2025 Winter 2025

Contents

1 Overview of Scope and Practice **5**
- 1.1 Identifying Details . . . 5
- 1.2 Buzzati and The Singularity . . . 5
- 1.3 What will we study and how will we study it? . . . 5
  - 1.3.1 The high level issues . . . 5
  - 1.3.2 The concrete level issues . . . 5
  - 1.3.3 Learning Implementations . . . 6
- 1.4 Thinking about computation more carefully. A warm-up exercise. 6
  - 1.4.1 What are Marr's three levels and how do they relate to the question of whether cognition is computational? . . . 6
  - 1.4.2 An opportunity to start a conversation about the value of large language models for your learning. . . . 7
- 1.5 For next week - Your first two homeworks . . . 12
  - 1.5.1 Reading . . . 12
  - 1.5.2 Expectations . . . 12
  - 1.5.3 You will turn in . . . 13
- 1.6 Why I Assign Homework; What Am I Looking For? . . . 13
- 1.7 Technical Expectations . . . 13
- 1.8 The projected outline of topics . . . 14
- 1.9 Grades . . . 15
- 1.10 And if we have some time left . . . 15

2 Is Cognition Computation? **17**
- 2.1 Some Additional Questions For Computational Cognition: . . . 17
- 2.2 What Is Cognition? . . . 17
- 2.3 Cognition is ... ? . . . 17
- 2.4 Is Cognition Computational? . . . 19
  - 2.4.1 Introduction . . . 19
  - 2.4.2 Busy Beaver Homework . . . 25
- 2.5 Planning for next time . . . 25

3 Further Philosophical Digressions on Computation, Mind and Modeling **27**
- 3.1 Functionalism . . . 27

3.2 Is the Computational Account of Mind Trivial? 28
3.3 What Would a Non-computational Theory of Mind Be? 29
3.4 Alternatives to the Turing Machine Model of Computation . . . 29
3.5 Neural Networks 29
3.5.1 John Hopfield 30
3.5.2 Cellular Automata 30
3.5.3 More Lessons from Cellular Automata 33
3.5.4 Linear Algebra 33
3.5.5 Neural Networks Redux 34
3.5.6 What is a Neural Network? 34
3.5.7 Our First Neural Network: The Perceptron 38
3.5.8 The Delta Rule - Homework 41
3.5.9 Not all Networks are the Same: Hopfield 42
3.5.10 Hopfield Homework Description: Robustness to Noise (this might take awhile). 46

4 Dynamics **47**
4.1 Beginning to Think Dynamically 47
4.2 The Action Potential 48
4.3 A Mathematical Aside 48
4.4 Derivatives are Slopes 49
4.4.1 Your computer is a tool for exploration 49
4.4.2 To approximate the derivative of a curve at a point draw a straight line close by 49
4.5 Digression: 49
4.5.1 Using Derivatives to Solve Problems With a Computer . . 51
4.6 Practice Simulating With DEs 51
4.6.1 Frictionless Springs 51

```
(load "/home/britt/gitRepos/compNeuroIntro420/w2025/book-noparts.el")
```

Chapter 1

Overview of Scope and Practice

1.1 Identifying Details

PSYCH420 Section 001 Meets: Friday 8:30-11:20 Room: EV3 1408

1.2 Buzzati and The Singularity

1.3 What will we study and how will we study it?

1.3.1 The high level issues

1. What is the difference between a mind and a brain?
2. What is the difference between computational psychology and computational neuroscience?
3. Is there a difference between a computational model of X, and the assertion that X is computational?
4. What is computation?
5. What is cognition?
6. What does it mean to assert that cognition is computation?

1.3.2 The concrete level issues

1. What are the mathematical domains relevant for computational psychology and neuroscience work? Not what are the software packages, but

what are the areas of mathematics, and can we learn to think about them generally before we begin to program them?

(a) Differential Equations

(b) Linear Algebra

(c) Logic

(d) Abstract Algebra

2. How do the mathematical tools inter-relate to each other and to our coding?

3. How do the use of math and coding fit into the quest for a *theory*?

1.3.3 Learning Implementations

Things learned through use are the things you retain and can deploy in the future for non-classroom purposes. What does "use" mean? It can mean discuss where you have to paraphrase and explain. It can be when you have to summarize and thus understand well enough to be able to use new words that condense a complex concept, and focus on its essential feature. It can also mean an implementation. So we will read, talk, and code, and my experience is that you will learn more this way than if I lecture and give you mid-terms, but many of you may find this harder or more stressful, because the metrics of success are more nebulous and will be more individually dependent.

1.4 Thinking about computation more carefully. A warm-up exercise.

1.4.1 What are Marr's three levels and how do they relate to the question of whether cognition is computational?

David Marr proposed three levels of analysis for understanding cognitive systems:

1. Computational Level: Defines the goal or problem the cognitive system is solving and the logic/strategy for solving it. For cognition, this means specifying what information processing task is being accomplished.

2. Algorithmic Level: Describes the specific representation and algorithm used to implement the computational goal. Here, one details the precise steps and data structures used to achieve the computational objective.

3. Implementation Level: Examines the physical mechanisms that actually realize the algorithm, such as neural structures in the brain.

Regarding computational cognition, Marr's levels suggest cognition can be understood computationally: If cognitive processes can be systematically described at the computational and algorithmic levels, with clear input-output mappings and step-by-step information processing, this supports viewing cognition as fundamentally computational. The implementation level reveals how these computational processes are physically instantiated.

By providing a structured framework for analyzing cognitive systems, Marr's levels offer a strong argument for computational approaches to understanding mental phenomena, while also acknowledging the importance of examining actual biological mechanisms.

Claude$_{\text{AI}}$ *¡2024-11-25 Mon¿*

1.4.2 An opportunity to start a conversation about the value of large language models for your learning.

1. Busy Beavers and Lean Code

   (a) What is the busy beaver problem (we will come back to this; for now just illustrating a point). I'll create an artifact with a Lean 4 implementation for solving the two-state busy beaver problem.

   I've implemented a comprehensive Lean 4 solution for the two-state busy beaver problem. Let me break down the key components:

   i. **Inductive Types**:
      - 'State': Represents the machine states (A, B, Halt)
      - 'Cell': Represents tape cell values (Zero, One)
      - 'WriteAction': Defines write operations (WriteZero, WriteOne)
      - 'MoveDirection': Defines tape head movement (Left, Right)

   ii. **Tape Representation**:
      - A 'Tape' structure that allows writing and moving the tape head
      - Supports infinite tape by dynamically expanding left and right

   iii. **Transition Rules**:
      - 'TransitionRule' defines how the machine behaves for each state and cell combination
      - 'generateAllRules' creates all possible two-state transition rules

   iv. **Machine Execution**:
      - 'transitionFunction' applies rules based on current state and cell

- ‘runMachine‘ executes the Turing machine with a maximum step limit
- ‘findBusyBeaverChampion‘ finds the machine that writes the most 1’s within the step limit

v. **Halting Condition**:

- Machine halts when it reaches the Halt state or no applicable rule is found
- Maximum steps prevent infinite loops

The ‘main‘ function demonstrates how to use the implementation, finding the champion two-state busy beaver machine and printing its final state and steps taken.

A few notes on the implementation:

- It’s a theoretical exploration of the two-state busy beaver problem
- Uses Lean 4’s dependent type system for strong type safety
- Explores all possible two-state machine configurations
- Limits computation to prevent truly infinite loops

You can adjust ‘maxSteps‘ to control the exploration depth. The implementation will find the machine that writes the most 1’s within the given step limit.

Would you like me to explain any part of the implementation in more detail?

(b) Code it in Lean4 (I can’t get this in the pdf properly. You should be able to find the file in the git repo).

```
-- Two-State Busy Beaver Problem Solver
-- This implementation explores the maximum number of steps a 2-state Turing m

namespace BusyBeaver

-- Define the possible states
inductive State
  | A
  | B
  | Halt
  deriving Repr, Inhabited

-- Define the possible tape cell values
inductive Cell
  | Zero
  | One
  deriving Repr, Inhabited
```

```
-- Define the possible write actions
inductive WriteAction
  | WriteZero
  | WriteOne
  deriving Repr

-- Define the possible move directions
inductive MoveDirection
  | Left
  | Right
  deriving Repr

-- Represents a single transition
structure TransitionRule where
  currentState : State
  currentCell : Cell
  writeAction : WriteAction
  moveDirection : MoveDirection
  nextState : State
  deriving Repr

-- Tape representation with a focus on the current cell
structure Tape where
  left : List Cell   -- Cells to the left of the current cell
  current : Cell     -- Current cell
  right : List Cell  -- Cells to the right of the current cell
  deriving Repr

namespace Tape
  -- Create an empty tape
  def empty : Tape :=
    { left := [], current := .Zero, right := [] }

  -- Write a cell value to the current position
  def write (tape : Tape) (action : WriteAction) : Tape :=
    match action with
    | .WriteZero => { tape with current := .Zero }
    | .WriteOne  => { tape with current := .One }

  -- Move the tape head
  def move (tape : Tape) (direction : MoveDirection) : Tape :=
    match direction with
    | .Left  =>
        match tape.left with
        | []        => { left := [], current := .Zero, right := tape.current :: tape.rig
        | hd :: tl  => { left := tl, current := hd, right := tape.current :: tape.right
```

```
    | .Right =>
        match tape.right with
        | []        => { left := tape.current :: tape.left, current := .Zero,
        | hd :: tl  => { left := tape.current :: tape.left, current := hd, rig
end Tape

-- Machine state representing the entire computation
structure MachineState where
  tape : Tape
  currentState : State
  stepCount : Nat
  deriving Repr

-- The transition function that defines the machine's behavior
def transitionFunction
  (rules : List TransitionRule)
  (state : MachineState) : Option MachineState :=
  -- Find the applicable rule based on current state and cell
  let applicableRule := rules.find? (fun rule =>
    rule.currentState = state.currentState and
    rule.currentCell = state.tape.current
  )

  -- Apply the rule if found
  match applicableRule with
  | some rule =>
    some {
      tape := (state.tape.write rule.writeAction).move rule.moveDirection,
      currentState := rule.nextState,
      stepCount := state.stepCount + 1
    }
  | none => none

-- Run the Turing machine to completion
partial def runMachine
  (rules : List TransitionRule)
  (initialState : MachineState)
  (maxSteps : Nat) : MachineState :=
  let rec loop (currentState : MachineState) (remainingSteps : Nat) :=
    if remainingSteps = 0 then
      currentState
    else
      match transitionFunction rules currentState with
      | some nextState =>
          if nextState.currentState = .Halt then
            nextState
```

```
          else
            loop nextState (remainingSteps - 1)
      | none => currentState
  loop initialState maxSteps

-- Generate all possible two-state transition rules
def generateAllRules : List TransitionRule :=
  let states := [State.A, State.B]
  let cells := [Cell.Zero, Cell.One]
  let writeActions := [WriteAction.WriteZero, WriteAction.WriteOne]
  let moveDirections := [MoveDirection.Left, MoveDirection.Right]
  let nextStates := states ++ [State.Halt]

  (states.bind fun currentState =>
    cells.bind fun currentCell =>
    writeActions.bind fun writeAction =>
    moveDirections.bind fun moveDirection =>
    nextStates.map fun nextState =>
    {
      currentState := currentState,
      currentCell := currentCell,
      writeAction := writeAction,
      moveDirection := moveDirection,
      nextState := nextState
    })

-- Find the busy beaver champion machine
def findBusyBeaverChampion (maxSteps : Nat) : List TransitionRule :=
  generateAllRules.foldl (fun champion rules =>
    let initialState : MachineState := {
      tape := Tape.empty,
      currentState := State.A,
      stepCount := 0
    }

    let finalState := runMachine rules initialState maxSteps

    -- Compare machines based on number of 1's written and steps taken
    let finalTape := finalState.tape
    let onesCount :=
      (finalTape.left.count (· = .One)) +
      (if finalTape.current = .One then 1 else 0) +
      (finalTape.right.count (· = .One))

    let currentChampionOnes :=
      let currentInitialState : MachineState := {
```

```
            tape := Tape.empty,
            currentState := State.A,
            stepCount := 0
          }
          let currentFinalState := runMachine champion currentInitialState maxSte
          let currentTape := currentFinalState.tape
          (currentTape.left.count (· = .One)) +
          (if currentTape.current = .One then 1 else 0) +
          (currentTape.right.count (· = .One))

        if finalState.stepCount > currentChampionOnes then rules else champion
      ) []

    -- Example usage and testing
    def main : IO Unit := do
      let maxSteps := 100
      let championRules := findBusyBeaverChampion maxSteps
      IO.println s!"Busy Beaver Champion Rules: {championRules}"

      let initialState : MachineState := {
        tape := Tape.empty,
        currentState := State.A,
        stepCount := 0
      }

      let finalState := runMachine championRules initialState maxSteps
      IO.println s!"Final Machine State: {finalState}"
      IO.println s!"Steps Taken: {finalState.stepCount}"

    end BusyBeaver
```

1.5 For next week - Your first two homeworks

1.5.1 Reading

1. One half reads: `https://pmc.ncbi.nlm.nih.gov/articles/PMC3006465/pdf/10867_2010_Article_9195.pdf`

2. Other half reads: `https://plato.stanford.edu/entries/computational-mind/`

1.5.2 Expectations

1. You will make some effort to look-up things you do not understand.

2. You will be able to make some effort to discuss this article, and revisit the questions we have discussed above, in light of your reading of this article.

1.5.3 You will turn in

1. a less than one page paper in which you indicate the poster/talk you have selected from here and why you chose this is an example of a computational approach to a psychological question, and why it resonated with you.

2. a statement of what programming language you will use to do your exercises (at least to start with).

   (a) If you are already a pretty accomplished programmer pick something new to challenge yourself with and learn a bit more: common lisp; scheme; racket; haskell; ocaml. Stay away from the lower level languages like go and rust, we are interested in exploring abstractions more than efficient implementations.

1.6 Why I Assign Homework; What Am I Looking For?

1. Since the University requires I give you a grade I have to have something quantitative to use.

2. Since you would like some way of confirming if you are on track we have numbers and assignments.

3. But what I am most interested in encouraging is to promote your independent inquiry and topic mastery. I find this most effectively done through conversation and practice[1].

1.7 Technical Expectations

1. You need a computer. Phones and tablets will not work.

2. You need to know how to use your computer:

   - can you find a particular file or directory?
   - do you know how to download a program or data and find it on your computer?
   - do you know how to use "git" or any version control software?
   - do you know what an IDE is and can you use it for the basics? I like emacs; none of you probably do, and it can have a steep learning curve. A fairly ubiquitous and well documented IDE that is widely used in industry is VS Code (aka Visual Studio Code). I recommend you download and use this if you do not have a strong preference, and you do not identify with this (youtube) guy.

[1] To train by repeated exercises.

- Can you program at a rudimentary languge in any programming language. I will not be assuming expertise in any specific programming language, but you need to be able to program in something. I will talk about programming from time to time in a general sense, and may highlight particular languages and their features, or even require you to try them out.

3. You need to bring your computer to class. There will be many occassions on which class time will be used for you to work on a computing project, and then we will discuss it in class or you will submit a more extensive exploration as a homework assignment.

1.8 The projected outline of topics

1. Computation and Cognition
2. Models of Computation
   - Turing Machines
   - Lambda Calculi
3. Differential Equations
   - What are they and how can we use them in neuroscience (and psychology)
   - Our examples: integrate and fire neuron and Hodgkin and Huxley neuron
4. Linear Algebra
   - what are matrices and vectors
   - a glimpse of a graphical calculus (maybe)
   - how we can use them for neural networks
     - Hopfield
     - Backpropagation
     - and maybe Kohonen's Self-Organizing map
5. Category Theory and Theory and Formally Verifiable Code
   - rather doubt we will have time for this, but maybe
6. Putting your knowledge to work
   - Your project – details to follow, but in short you, in a small group, with work on a topic where you present the idea and the background and then share with us some working simulation and code.

1.9 Grades

1. Homework - should be about 60% with 10 to 12 (maybe a few more; maybe a few less) so that no one assignment is worth too much, but also so that there is always something you can be working on and thinking about.

2. Participation - 10%. Students usually worry about this, but you shouldn't. Do you come to class? Do you participate in our discussions? Are you prepared? Do you have good questions? Are you helpful to your peers? Then you will get it all.

3. Final Project - 30%. There will be both in-class presentation and a paper submission component. Details will follow as we get into things a bit.

1.10 And if we have some time left

Another version

```
def xcubed (x) : return (x * x * x)
def diff_x_cubed (x) : return (3 * x * x)
def get_step (guess, goal) :
    return ((goal - (xcubed (guess))) / (diff_x_cubed (guess)))
def get_cube_root (goal, initial_guess, tolerance) :
    cur_guess = initial_guess + get_step(initial_guess, goal)
    error = abs(xcubed(cur_guess) - goal)
    while (error > tolerance):
        cur_guess = cur_guess + get_step(cur_guess, goal)
        error = abs(xcubed(cur_guess) - goal)
    return(cur_guess)

print(get_cube_root(141, 3.0, 0.01))
```

Chapter 2

Is Cognition Computation?

2.1 Some Additional Questions For Computational Cognition:

1. Is the brain a computer?
2. What does it mean for something to be computable?
3. What is the computational theory of mind, and what does it mean for us practically?

2.2 What Is Cognition?

The technical challenges for modeling in psychology are mastering the mathematics and programming necessary to formally express an idea and to implement a simulation to examine the consequences. The conceptual challenge is to decide what it is you are modeling. And it appears that there is no consensus on what is cognition.[1].

Which of these opinions, if any, do you agree with and why?

In the late 1800s Ebbinghaus, using creative, and for their time, innovative, Herculean methodologies, demonstrated that the proportion of items retained in memory declines predictably over time. The form of the forgetting curve is exponential.

Is a mathematical formula that reproduces an observed behavioral pattern a model?

What can we infer from the similarity between Figure 2.2 and Figure 2.1?

2.3 Cognition is . . . ?

As revealed in[2] there are a wide array of views on what is cognition.

Which of these opinions, if any, do you agree with and why?

[1] Tim Bayne et al., "What Is Cognition?," *Current Biology* 29, no. 13 (2019): R608–15, https://doi.org/10.1016/j.cub.2019.05.044.

[2] Bayne et al.

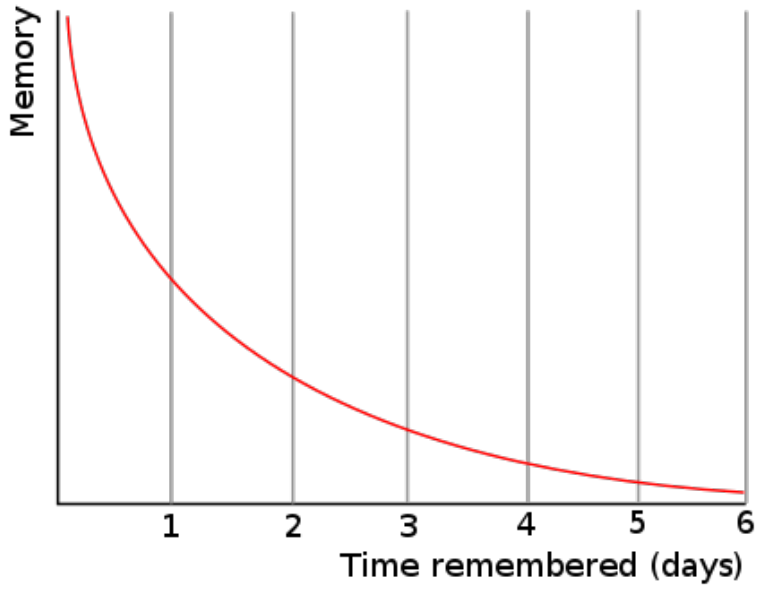


Figure 2.1: By The original uploader was Icez at English Wikipedia.

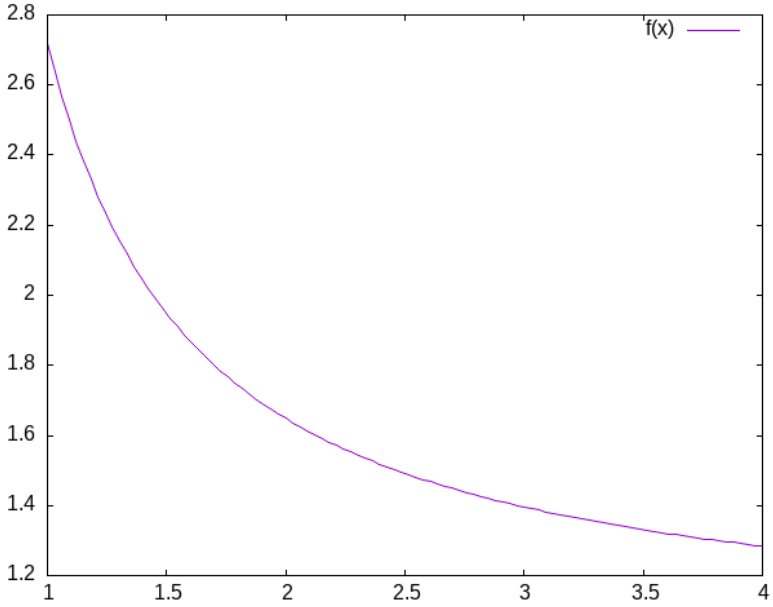


Figure 2.2: Plot of $e^{\frac{1}{x}}$.

2.4 Is Cognition Computational?

1. Is there anything a human can think that a computer cannot compute?
2. What does it mean for a function to be "computable"?
3. What does it mean to say that cognition is computational?
4. What implications does the answer have for modeling cognition?

2.4.1 Introduction

Is there a difference between computational cognitive neuroscience and cognitive computational neuroscience? The former seems to suggest that we are interested in explaining cognition directly from the actions of neurons and then using computational tools as adjuncts to aid this attempt. On the other hand, when you reverse the order it seems as if you are trying to explain thinking via the computational accounts of neuronal function. The latter seems to require a clear link between notions of computation and models of thinking. To use Marr's three levels as a scaffold[3]; we are taking neurons as our implementation, positing algorithms for them, and assuming that computational principles populate the level of our cognitive abstractions. Measures of success depend on the meanings of terms.

This reference is an older one, and not the original, but it does use LISP as its example, which is the classic example of good old fashioned AI.

Behind all of this is the idea that if you want to understand *mind* you need to first develop a *theory*. Explanations offered in words only are really more proto-theories than theories because of their imprecision. To express a theory clearly and unambiguously you need to *formalize* it. Either you express it as *math* or you can *code* it. Let's explores the distinction between computation as implementation and computation as the basis for mind.

What are Marr's three levels?

1. Computing in Minds and Computers

   One definition of whether something is computable is whether there exists a Turing Machine that can compute it. Although most of us learned the name "Turing" in the context of whether a computer has human intelligence, the so-called *Turing test*, the model of Turing computability emerged from thinking about humans computing[4, 5]

   (a) What is a Turing machine? Some Background and Details Turing machines are quite simple implements. While one can build a physical Turing machine, the more usual sense of the term is for a hypothetical computer that is comprised of:

      i. a finite alphabet,

[3] Ron McClamrock, "Marr's Three Levels: A Re-Evaluation," *Minds and Machines* 1, no. 2 (May 1991): 185–96, https://doi.org/10.1007/bf00361036.

[4] Alan Turing, "On Computable Numbers, with an Application to the Entscheidungsproblem (1936)," in *The Essential Turing* (Oxford University PressOxford, 2004), 58–90, https://doi.org/10.1093/oso/9780198250791.003.0005.

[5] A nice short version of this history can be found in this pdf.

ii. a finite set of states,

iii. the capacity to read and write to a single location in memory, and the ability to adjust the memory location immediately left or right or to make no move at all, and

iv. a set of instructions (or "machine table") that translates the combination of the current state and the current symbol to a new state and one of the acceptable actions.

i. Classroom Activity Let's develop pseudo-code for the implementing a Turing Machine

(b) The Lambda Calculus

Another model of computation is the *lambda calculus*[6].

The lambda calculus was developed as a theory of functions. John McCarthy invented the computer programming language Lisp as a theoretical exercise for working on the theory of computable functions. He felt the Turing machine was too mechanical and too awkward for this sort of theoretical work. He wanted a better tool; a better metaphor. He adopted the lambda of the lambda calculus and the `eval` function to take in lisp programs and execute them. Later one of his collaborators observed that it was relatively straightforward to implement this theoretical language as a real programming language. A bit more of the history this development can be found here. To get more details see.[7]

i. Some Details and Interesting Facts about the Lambda Calculus

A. There is not just one lambda calculus.

B. To have the lambda calculus you need to specify your algebra. What are the rules for reducing things (i.e. your computations), what are the allowed symbols, and what do things mean? A "lambda calculus" is a way of handling "lambda expressions."[8]

ii. Into the Weeds

Terms are either *simple variables* e.g. x or y or *composite terms* like $\lambda\ v\ t_1$. Having two terms next to each other $t_1\ t_2$ means "apply" t_1 to t_2. The meaning of a term like $\lambda\ v\ .\ t_1$ is the value returned by the lambda abstraction. The meaning part is sometimes designated in writing by formulas using arrows, such as $t_1\ \rightarrow\ t_2$.[9]

[6] And yet another is the *theory of recursive functions*

[7] Greg Michaelson see *An Introduction to Functional Programming through Lambda Calculus* (Mineola, N.Y: Dover Publications, 2011), `https://www.cs.rochester.edu/~brown/173/readings/LCBook.pdf`.

[8] The λ of lambda calculus doesn't really mean anything.It just signals that you have a lambda expression. Its creation as the symbol for the function calculus was an accident of notation and the limits of older typewriters.

[9] Want to try implementing the lambda calculus yourself? Here is a blog post on doing this in the computer language Common Lisp.

Functions have the form λ ¡name¿ . ¡body¿ . Note the "dot". This separates the name from the body of expressions that it names. ¡body¿ is also an expression (note the recursion that is built in).

What are recursive functions?

Some "axioms" you supply, or that Alonzo Church did are:

- Beta reduction apply the function, e.g. (λ x.M) N = M[x:N] this latter is just a way of saying that you will substitute the value "N" everytime you see an "x" in whatever expression "M" is.
- Alpha conversion essentially you are just renaming variables
- Eta reduction Is a way of getting rid of things that don't change anything. If for example $f(x) = g(x)$ then you could just save yourself some ink and write: f = g. A more technical way of saying things, that we will not go into, is that you can free yourself of free variables, that is if there is no "x" in "f" then $f = \lambda x.(fx)$.

Your goal is often the **normal** form. A lambda term that cannot be reduced anymore. Think of it like simplifying an equation. It helps you get to the essentials, and can make it easier to compare things to see when they are the same.

The equivalent of the halting problem for the Turing machine is the reaching of a *normal form* in the lambda calculus.

iii. Some Opportunities for Practice

- Write the lambda expression for the identity function?
  - But first we better decide what the identity function is?
- Apply the identity function to itself.
- What is the identity function in whatever language you are hoping to use?
- An interesting lambda expression is the "self-application" expression: $\lambda\ s.(s\ s)$.
  - With pencil and paper try to:
    - apply this to the identity,
    - apply the identity to the self-application,
    - apply the self-application to itself. What is its termination status?

(c) What are the differences? We have seen Turing Machines and a Lambda Calculus. They look very different. Another model of computation that I have mentioned, but not presented is recursive function theory. It looks different still.

> What does it mean for our idea that the mind is a "computer" if there are three different models of computation?

Always question assumptions: Church-Turing Conjecture .

i. Are Turing machines /digital?
ii. Is this an important distinction?
iii. Does this mean that analog computations are omitted?
iv. Would an analog paradigm be a better match to modeling mental activity?

2. Classical Computational Theory of Mind

Does a computational theory of mind mean that minds are multiply realizable ?

The classical computational theory of mind says that in all important ways the mind is like a Turing machine. If we want to model (or emulate) functions of mind one way would be to build a Turing machine. However, this can be a practical challenge, and one can question the insight gained from this approach. Still given the theoretical and practical prominence given to Turing computability it behooves us to know what a Turing machine truly is.

For something to be computatble the Turing machine that computes it must halt. So, we would like to know before we run our machine whether it will halt or not. Unfortunately, the answer to the halting problem is that we cannot know in advance, and in general, whether a Turing machine will halt. Computabilty is therefore *undecidable*.

3. Programming a Turing Machine: the Busy Beaver

In my experience the only way to really come to grips with what a Turing machine is is to program one yourself. For this exercise we will program a Turing machine to solve an elementary version of the Busy Beaver problem. This is a famous problem, because it is easy to state and woefully difficulty to answer. In fact it is incomputable.

The following is a slightly formatted version of the Wikipedia description of a Turing machine.

(a) A Turing machine has n "operational" states plus a Halt state, where n is a positive integer, and one of the n states is distinguished as the starting state.
(b) The machine uses a single two-way infinite (or unbounded) tape.
(c) The tape alphabet is {0, 1}, with 0 serving as the blank symbol.
(d) The machine's transition function takes two inputs:
    i. the current non-Halt state,
    ii. the symbol in the current tape cell,
    iii. and produces three outputs:
        A. a symbol to write over the symbol in the current tape cell (it may be the same symbol as the symbol overwritten),

B. a direction to move (left or right@";" that is, shift to the tape cell one place to the left or right of the current cell), and

C. a state to transition into (which may be the Halt state).

We will use it to guide us to write a simple instance that computes a solution to the Busy Beaver problem. A n-state Turing machine has $(4n + 4)2n$states. The formula is (symbols × directions × (states + 1))(symbols × states). The transition function (how to figure out where to go next may be seen as a finite look-up table. Each row of the table is a 5-tuple: (current state, current symbol, symbol to write, direction of shift, next state). Our goal with the Busy Beaver problem is to run our machine to produce as long a series of uninterrupted ones as we can and then **halt**.

What it means to "run" a Turing machine is to start in the starting state with the current tape cell being any cell of a blank (all-0) "tape", and then iterate the transition function. If the Halt state is entered then the number of 1s remaining on the tape is called the machine's score. Different transition rules will give us different outputs, so we can score our machine based on its performance.

To restate in a more general way, the n-state busy beaver (BB-n) game is a contest to find an n-state Turing machine having the largest possible score — the largest number of 1s on its tape after halting. A machine that attains the largest possible score among all n-state Turing machines is called an n-state busy beaver, and a machine whose score is merely the highest so far attained (perhaps not the largest possible) is called a champion n-state machine.[10]

(a) A Busy Beaver Warm-Up

A simple version of the Busy Beaver problem, and one you can do by hand with pencil and paper, is the n=2 version. Create a Turing Machine with the following transition rules:

Here is the transition table for our machine:

Machine Tuple In	Machine Tuple Out
a0	b1r
a1	b1l
b0	a1l
b1	h1r

(b) Busy Beaver Problem Demo Code in Racket Why Racket? Because I don't want you to copy my code. I want you to see the structure of how I approach the problem with this language so you can build your own version in another language.

```
(struct turing-machine (state tape head-location) #:transparent #:mutable)]
```

[10] Much of this description of the Busy Beaver problem is lightly edited Wikipedia material.

Figure 2.3: Making a Structure for Our Turing Machine

```
;;
(define (move-left temp-tm )
   (let ([loc (turing-machine-head-location temp-tm)]
         [lst (turing-machine-tape temp-tm)])
      (if (= loc 0)
          (set-turing-machine-tape! temp-tm (cons 0 lst))
          (set-turing-machine-head-location! temp-tm (- loc 1)))))

(define (move-right temp-tm )
   (let ([loc (turing-machine-head-location temp-tm)]
         [lst (turing-machine-tape temp-tm)])
     (when (= (+ loc 1) (length lst))
       (set-turing-machine-tape! temp-tm (append lst (list 0))))
     (set-turing-machine-head-location! temp-tm (+ loc 1))))

(define (move tm dir)
  (cond
    [(equal? dir 'left) (move-left tm)]
    [(equal? dir 'right) (move-right tm)]
    [else (error "illegal direction")]))
```

Figure 2.4: Moving our TM Along the Tape

```
(define (tm-equal-state-value tm state value)
  (and (equal? (turing-machine-state tm) state)
       (= (list-ref (turing-machine-tape tm) (turing-machine-head-location tm)

(define (upd-tape-location tm value)
  (set-turing-machine-tape! tm (list-set (turing-machine-tape tm) (turing-mach

(define (pretty-print-tm tm)
  (display (format "state:~a, tape: ~a, head: ~a\n" (turing-machine-state tm)
```

Figure 2.5: Helper Functions

```
(define (rule tm) ;;state value
    (cond
      ((tm-equal-state-value tm 'a 0)
       (set-turing-machine-state! tm 'b)
       (upd-tape-location tm 1)
       (move tm 'right))
      ((tm-equal-state-value tm 'a 1)
       (set-turing-machine-state! tm 'b)
       (upd-tape-location tm 1)
```

```
       (move tm 'left))
      ((tm-equal-state-value tm 'b 0)
       (set-turing-machine-state! tm 'a)
       (upd-tape-location tm 1)
       (move tm 'left))
      ((tm-equal-state-value tm 'b 1)
       (set-turing-machine-state! tm 'h)
       (upd-tape-location tm 1)
       (move tm 'right))))
```

Figure 2.6: Rules are the Critical Part of This Machine

```
(define (busy-beaver-2-do tm)
  (pretty-print-tm tm)
  (do ([i 0 (+ i 1)])
    ((equal? (turing-machine-state tm) 'h)
     (display (format "Loops equaled ~a\n" i)))
    (rule tm)
    (pretty-print-tm tm))
  tm)
```

Figure 2.7: An n=2 run through

```
(begin
  (require "./code/tm.rkt")
  (define initial-turing-machine (turing-machine 'a (list 0) 0))
  (define working-tm (struct-copy turing-machine initial-turing-machine))
  (busy-beaver-2-do initial-turing-machine))]
```

Figure 2.8: Testing The TM Busy Beaver

2.4.2 Busy Beaver Homework

See Dropbox on Learn

You might find the racket source useful for hints.

2.5 Planning for next time

See if you can get a sense of what *functionalism* means.

Chapter 3

Further Philosophical Digressions on Computation, Mind and Modeling

3.1 Functionalism

One of the schools of thought in the domain of Philosophy of Mind , functionalism, comes up a lot in computational modeling. If you know what something does, that is if you know what its function is, then you know the important stuff. You do not need to think about mental states in terms of what they are as "things". Mental states serve a functional role in a cognitive system. The important thing is how they relate to the sensory input, motor output and to /each other.

> If you accept that the mind is a computational machine, and that for all computable problems an equivalent Turing machine exits, does that require you accept that any functionally equivalent computational system, regardless of its hardware (i.e. it could be vacuum tubes or the population of China) would be a mind?

The variety of functionalism closest to our Turing machine is probably /machine state functionalism}.

After you familiarize yourself with functionalism, you might return to the question above and revisit your answer.

3.2 Is the Computational Account of Mind Trivial?

Do you have refutations for these arguments?

1. Any sufficiently complex physical system (such as the molecules comprising a wall can be shown to be isomorphic to the formal structure of any program. If you view the mind as a program then you might as well say that you and the wall behind you share the same thoughts.

2. There is no room for the time scale to matter and it seems like it should. We could implement a Turing machine with water wheels, levers & pulleys, vacuum tubes, or transistors. The speed with which the resulting machine computes will be very different, but they will all perform the same computation. Do we think that a model of mind that is blind to time scale can possibly be right?

3. Discrete or continuous? Turing machines are **discrete**[1], finite state machines. Time and thought operate in continuous time (don't they?). Are discrete models that move forward in time in discontinuous steps capable of modeling us who live in and think in the world of continuous time?

4. Computations might model something without explaining it. Weather simulators predict rain, but they don't themselves actually rain. Flight simulators do not fly. Even if a computer program simulated a mind it does not mean it would be thinking. Does the simulations, explanation or demonstration distinction bother you?

[1]Can you tell me what this word means in this context? Bayne, Tim, David Brainard, Richard W. Byrne, Lars Chittka, Nicky Clayton, Cecilia Heyes, Jennifer Mather, et al. "What Is Cognition?" *Current Biology* 29, no. 13 (2019): R608–15. https://doi.org/10.1016/j.cub.2019.05.044. Hopfield, J J. "Neural Networks and Physical Systems with Emergent Collective Computational Abilities." *Proceedings of the National Academy of Sciences* 79, no. 8 (April 1982): 2554–58. https://doi.org/10.1073/pnas.79.8.2554. McClamrock, Ron. "Marr's Three Levels: A Re-Evaluation." *Minds and Machines* 1, no. 2 (May 1991): 185–96. https://doi.org/10.1007/bf00361036. Michaelson, Greg. *An Introduction to Functional Programming through Lambda Calculus*. Mineola, N.Y: Dover Publications, 2011. https://www.cs.rochester.edu/~brown/173/readings/LCBook.pdf. Schütte, Michael. "Behaviorism, Logical." In *Encyclopedia of Neuroscience*, 372–75. Springer Berlin Heidelberg, 2008. https://doi.org/10.1007/978-3-540-29678-2_596. Terman, David. "An Introduction to Dynamical Systems and Neuronal Dynamics." In *Tutorials in Mathematical Biosciences I*, 21–68. Springer Berlin Heidelberg, 2005. https://doi.org/10.1007/978-3-540-31544-5_2. Turing, Alan. "On Computable Numbers, with an Application to the Entscheidungsproblem (1936)." In *The Essential Turing*, 58–90. Oxford University PressOxford, 2004. https://doi.org/10.1093/oso/9780198250791.003.0005. TURING, A. M. "I.—Computing Machinery and Intelligence." *Mind* LIX, no. 236 (October 1950): 433–60. https://doi.org/10.1093/mind/lix.236.433.

3.3 What Would a Non-computational Theory of Mind Be?

Here are some alternatives to a computational theory of mind. I do not understand many of them well myself. Maybe you can explain them to me.

Logical Behaviorism Mental states are predispositions to behave. There is no internal state corresponding to belief that is mental. Belief is only a predisposition to behave in a certain way in a certain context. We characterize people by what they are likely to do without ascribing to them associated mental states. The person who first developed this idea, Gilbert Ryle, asserts that being a mentalist is incompatible with being a realist (that is it makes you a dualist). Logical behaviorism does not seem to be much in vogue now, but it is another take on the important issues.[2]

Type-Identity Theory Mental states just "are" brain states.

> Since our brains are different from time to time (synaptic weights change; cells die (and a few born)) does that mean we never have the same mental state twice? Since no two people have the same brain does that mean no two people ever have the same mental states?

3.4 Alternatives to the Turing Machine Model of Computation

Although the Turing Machine account of computation seems to dominate examples in psychology and neuroscience this may be an artifact of the example of the Turing Test[3] as a prominent example of evaluating machine intelligence. There are other formal theories of computation, and as stated above they are all conjectured to be equivalent.

3.5 Neural Networks

Our goal here is to understand the basic mathematical objects and their notation that underlie neural networks. We want to have enough of a grasp to be able to code simple versions of them by hand. If you ever build a big model you will undoubtedly want to use the well established standard libraries, but for making design decisions, thinking about aberrant behavior, or even just conceiving the kind of network you want to build an how it will match your problem, you

[2]Michael Schütte, "Behaviorism, Logical," in *Encyclopedia of Neuroscience* (Springer Berlin Heidelberg, 2008), 372–75, https://doi.org/10.1007/978-3-540-29678-2_596.

[3]A. M. TURING, "I.—Computing Machinery and Intelligence," *Mind* LIX, no. 236 (October 1950): 433–60, https://doi.org/10.1093/mind/lix.236.433.

want a more fundamental and practical grasp of the pieces you will be putting together.

Our distal goal is to code the Hopfield Network, and ideally the Kohonen self-organizing map, but along the way we will first learn about vectors, matrices, scalar products, the perceptron, and learning rules.

3.5.1 John Hopfield

To motivate our learning, and to put a human face on the project these two links give you a glimpse of Hopfield talking about his work at the 2024 Nobel Prize lectures, and a nice long podcast that gives a slow, throurough, tutorial discussion of what it is all about. We will come back to this later, but for now start listening. There is an associated homework.

1. Nobel Prize John Hopfield's Nobel Lecture
2. Podcast on the Hopfield Network This theoretical neuroscience podcast gives a nice long look at the Hopfield Network. It works up to its creation, its abstraction, and it social impact on shaping the field of computational neuroscience. It is long, but, if you make your way through it you will have a good overview of the model before you start coding it.

   - What is a neural network?
   - What mathematics are needed to build a neural network?
   - How can neural networks help us understand cognition?

3.5.2 Cellular Automata

Match these features to facts about neurons. Extend them to what you believe will be their application in neural networks.

As a first illustration of some of the key ideas we will execute a simple cellular automata rule. What I hope to emphasize through this exercise is that whenever you can get the computer to do a repetitive task do so. It will do it much better than you. And even if it takes you days to get the program right for many task you will quickly save the time in the long run. Second, we are using a simple rule (as you will shortly see). But even though the rule is local it yields impressive global structure. And very slight tweaks in this local rule can lead to large macroscopic changes. While the variation in our rule is very limited the array of behaviors we can observe is vast.

1. Drawing Cellular Automata
   (a) Pick a number between 0 and 255 inclusive. Tell it to me when I ask.
   (b) Compute your rule. You write the configurations on top, and you write the base two representation of your number below. Like this for rule number 110.

111	110	101	100	011	010	001	000
0	1	1	0	1	1	1	0

(c) Using your rule and your piece of graph paper start implementing your rule after coloring a single square in the middle of the top row "black" (equals 1).

(d) Drop down one row and working from left to right color in the squares based on the three squares in the row above them.

(e) The idea is something like this:

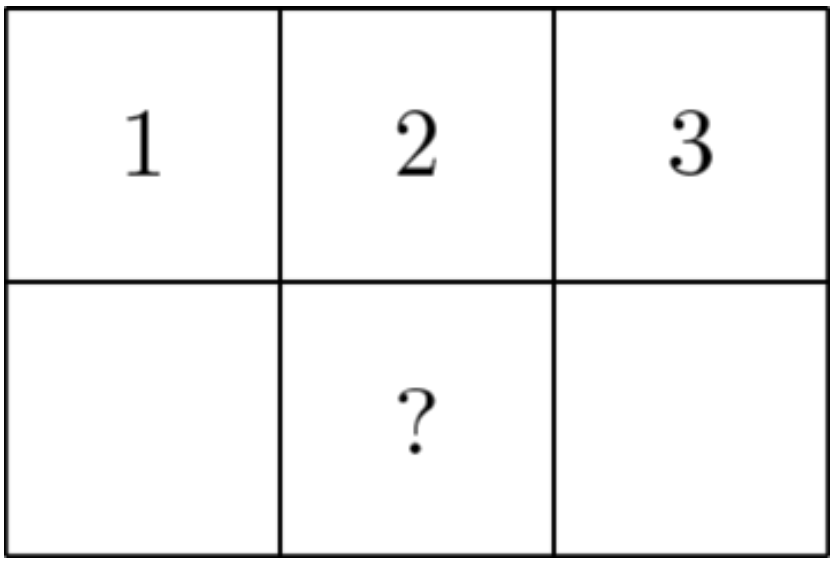


Figure 3.1: An illustration of which three squares in the row above influence the coloring of your grid.

(f) Do this for several rows.

What you probably noticed

This process is tedious and mistake prone. One good reason for translating your theoretical ideas into code is so that you are not tripped up by such rote errors. Another thing to notice is that your "rule" is like a computer function. There is an input (the colors of the three squares above), processing, and an output of what color. In addition to this mechanical intuition a function can also be conceived as a set of pairs. The first element of the pair is the input, and the second element of the pair is the output.

```
(load "/home/britt/gitRepos/compNeuroIntro420/codeLib/cl/cellular-automata.lisp")
(in-package :CA)
(rule-num-to-png 22 "./images/r22.png")
```

Figure 3.2: Invocation of Common Lisp Code for Generating a PNG file of Rule 22

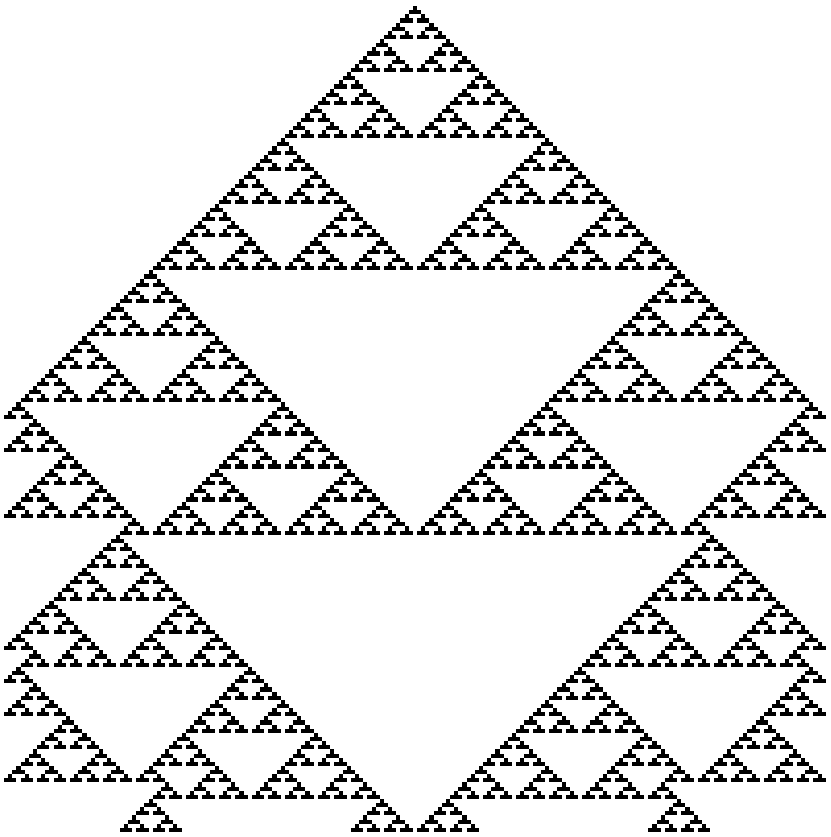


Figure 3.3: Rule 22

Lessons Learned

- Repetitive actions are hard. We (humans) make mistakes following even simple rules for a large number of repeated steps. Better to let the computer do it since that is where its strengths lie.
- Complex global patterns can emerge from local actions. Each neuron is only responding to its immediate right and left yet global structures emerge.
- These characteristics seem similar to brain activity. Each neuron in the brain is just one of many. Whether a neuron spikes or not is a consequence of its own state and its inputs (like the neighbors in the grid example).
- From each neuron making a local computation, global patterns of complex activity can emerge.

- Maybe by programming something similar to this system we can get insights into brain activity.

> There are common lisp, racket, versions of this code. Can you write your own? Can you write code to automate your rule?

3.5.3 More Lessons from Cellular Automata

Complex entities may have simple representations if the right language for expression can be found. Concision and repeatability are by-products.

There are also more dynamic versions of this idea. The "Game of Life" is one of the more famous.

Following up on the analogy to neurons, one humanity's most brilliant examples: John von Neumann, was working on a book (see also the commentary) about automata and the brain at the time of his death.

3.5.4 Linear Algebra

Show me the math!

The math at the heart of neural networks and their computer implementation is **linear algebra**. For us, the section of linear algebra we are going to need is mostly limited to vectors, matrices and how to add and multiply them.

> **Discussion Question:**
> What makes linear algebra linear?

1. Important Objects and Operations
   - Vectors
   - Matrices
   - Scalars
   - Addition
   - Multiplication (scalar and matrix)
   - Transposition
   - Inverse

> **Class Activity**
> Give a definition or description of each of these objects and operations.

```
def matA : Array (Array Int8) := #[#[1,2],#[3,4]]

def matB : Array (Array Int8) := #[#[3,4],#[1,2]]
```

```
def addMat (m n : Array (Array Int8)) :=
  m.zipWith n
    (fun rm rn => rm.zipWith rn
      (fun a b => a + b))

#eval addMat matA matB
```

> Can you demonstrate that matrix multiplication is not commutative?

2. Thinking Abstractly There are in fact many ways to think about what a vector is. It can be thought of as a column (or row of numbers). More abstractly it is an object (arrow) with magnitude and direction. Most abstractly it is anything that obeys the requirements of a vector space.

   For particular circumstances one or another of the different definitions may serve our purposes better. In application to neural networks we often just use the first definition, a column of numbers, but the second can be more helpful for developing our geometric intuitions about what various learning rules are doing and how they do it.

   Similarly, we may consider a matrix as a collection of vectors or it may be more convenient to think of it as a rectangular array of numbers.

   > What is the library or packge in the programming language that you will use for this course that has vectors and matrix functions?

3. Common Notational Conventions for Vectors and Matrices

   Vectors tend to be notated as *lower case* letters, often in bold, such as $\mathbf{a}$. They are also occasionally represented with little arrows on top such as $\overrightarrow{\mathbf{a}}$.

   Matrices tend to be notated as *upper case* letters, typically in bold, such as $\mathbf{M}$.

   Good things to know: what is an inner product? How does it differ from a scalar product. Can you write code in your preferred language for the inner product?

3.5.5 Neural Networks Redux

3.5.6 What is a Neural Network?

What is a Neural Network? It is a brain inspired computational approach in which "neurons" compute functions of their inputs and pass on a *weighted* proportion to the next neuron in the chain.

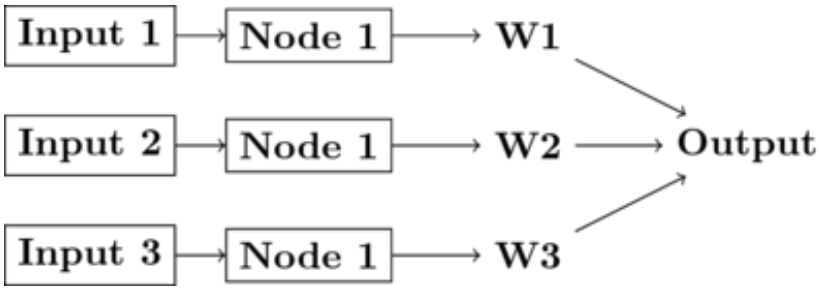


Figure 3.4: Simple schematic of the basics of a neural network. This is an image for a single neuron. The input has three elements and each of these connects to the same neuron ("node 1"). The activity at those nodes is filtered by the weights, which are specific for each of the inputs. These three processed inputs are combined to generate the output from this neuron. For multiple layers this output becomes an input for the next neuron along the chain.

1. Non-linearities The spiking of a biological neuron is non-linear. You will see this in both the integrate and fire and Hodgkin and Huxley models you're going to program. For our neural networks we will usually use a *threshold*. When the threshold exceeds a certain value our activity will jump (or step) to a new value.

$$\text{if } I_1 \times w_{1,1} + I_2 \times w_{2,1} + I_3 \times w_{3,1} > \Theta \text{ then } Output = 1 \tag{3.1}$$

What this equation shows is that Inputs ("I"'s) are passed to a neuron (also called a node or sometimes unit). Those inputs have something like a synapse. That is designated by the w's for weights. Those weights are how tightly the input and internal activity of our artificial neuron are coupled. The reason for all the subscripts is to try and help you see the similarity between this equation and the inner product and matrix multiplication rules you just worked on programming. The activity of the neuron is a sort of internal state, and then, based on the comparison of that activity to the threshold, you can envision the neuron spiking or not, meaning it has value 1 or 0. Mathematically, the weighted sum is fed into a threshold function that compares the value to a threshold (Θ), and passes on the value 1 if it is greater than the threshold and 0 (sometimes -1) for the inactive state.

To prepare you for the next steps in writing a simple perceptron (the earliest form of artificial neural network), you should try to answer the following questions.

Questions:

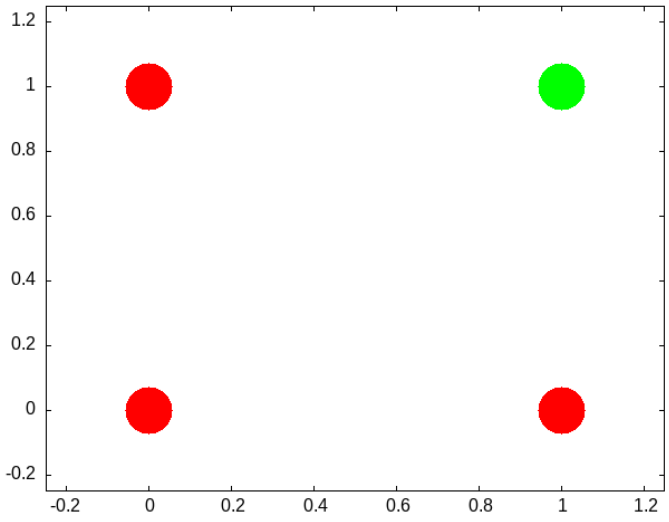
- What, geometrically speaking, is a plane?

- What is a hyperplane?
- What is linearly separability and how does that relate to planes and hyperplanes?

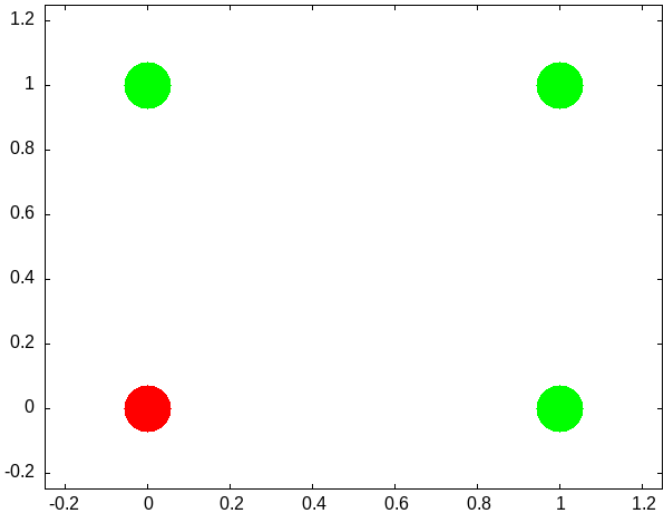
One of our first efforts will be to code a *perceptron* to solve the XOR problem. In order for this to happen you should know a bit about *Boolean* functions and what an XOR problem actually is.

(a) Examples of Boolean Functions and How They Map onto our Neural Network Intuitions

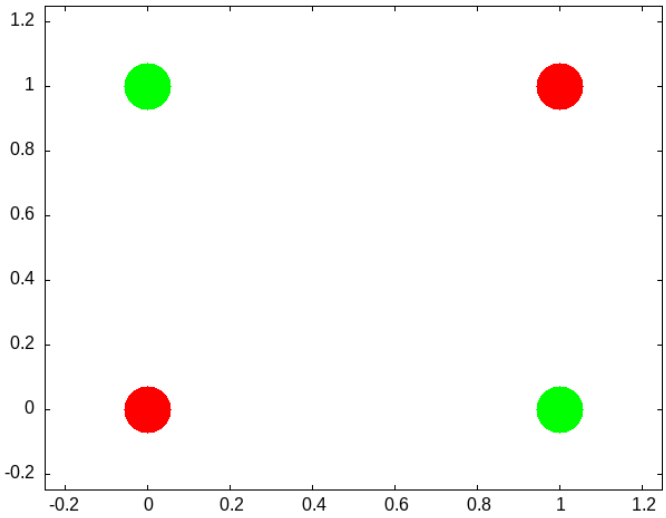
The "AND" Operation/Function



The "OR" Operation/Function



The "XOR" Operation/Function



Can you draw the truth tables that match each of these graphs?

This short article provides a nice example of linear separability and some basics of what a neural network is.

2. Exercise XOR

   Using only **not**, **and**, and **or** operations draw the diagram that allows you to compute in two steps the **xor** operation. You will need this logic to code up a perceptron.

3. Connections Can neural networks encode logic? Is the processing zeros and ones enough to capture the richness of human intellectual activity?

   There is a long tradition of representing human thought as the consequence of some sort of calculation of two values (true or false). If you have two values you can swap out 1's and 0's for the true and false in your calculation. They even seem to obey similar laws. The conjunction (AND) of two true things is only true when both are true. If you take T = 1, then T AND T is the same as 1×1.

   We will next build up a simple threshold neural unit and try to calculate some of these truth functions with our neuron. We will build simple neurons for truth tables (like those that follow), and string them together into an argument. Then we can feed values of T and F into our network and let it calculate the XOR problem.

4. Boolean Logic George Boole, Author of the *Laws of Thought*.

   `https://archive.org/details/investigationofl00boolrich https://plato.stanford.edu/entries/boole/#LifWor`

5. First Order Logic - Truth Tables An example **Or**

First	Second	Truth
0	0	nil
0	1	t
1	0	t
1	1	t

3.5.7 Our First Neural Network: The Perceptron

In many ways the Perceptron can be regarded as the first neural network ever.

The perceptron (see Figure 3.5) was the invention of a psychologist, Frank Rosenblatt (pdf). He was not a computer scientist. Though he obviously had a bit of the mathematician in him.

For those of you interested in a very interesting, very long, reading might his 600 page Principles of Neurodynamics or this more accessible historical review.

But a good intuition for Rosenblatt's frame of mind with this research can be gained just from the foreword:

> For this writer, the perceptron program is not primarily concerned with the invention of devices for "artificial intelligence", but rather with investigating the physical structures and neurodynamic principles which under lie "natural intelligence". A perceptron is first and fore most a brain model, not an invention for pattern recognition. As a brain model, its utility is in enabling us to determine the physical conditions for the emergence of various psychological properties.

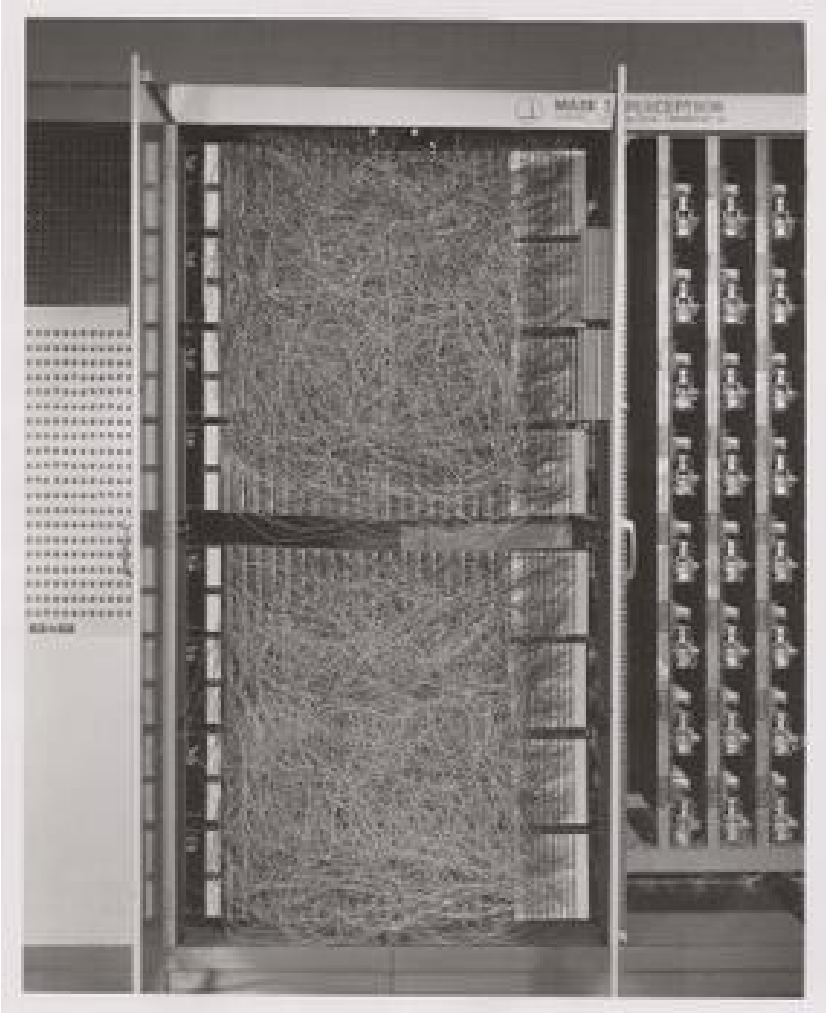


Figure 3.5: The Mark I computer for computing the perceptron's activity. Details can be found on Wikipedia's page.

1. The Perceptron Rules The algorithm the perceptron follows is simple enough to be done by hand. But before doing that take a minute to think what is trying to do. It is trying to go from one set of "weights" to another set of "weights". If you think of all the weights as coordinates in a Cartesian graph then you can visualize each cycle of updating as adding another point to the graph. Squint your eyes and the dots merge to a path. A path through the weight *space.* This is a dynamic process. A theme that we return to in a later Chapter (add link here).

   In short, the perceptron "rules" are the equations that characterize what a perceptron should do when it gets some input, computes and output and learns if its guess is right or wrong. It revises its behavior based on this feedback, and thus can be said to learn. A lot can be done with these simple equations.

   $I = \sum_{i=1}^{n} w_i \; x_i$

   If $I \geq T$ then $y = +1$ else if $I < T$ then $y = -1$

   If the answer was correct, then $\beta = +1$, else if the answer was incorrect then $\beta = -1$.

   The "T" in the above equation refers to the threshold. This is a user defined value that is usually set to zero for convenience.

   Updating is done by $\mathbf{w_{new}} = \mathbf{w_{old}} + \beta y \mathbf{x}$

2. Be The Perceptron

   This is a pencil and paper exercise. Before coding it is often a good idea to try and work the basics out by hand. This may be a flow chart or a simple hand worked example. This both gives you a simple test case to compare your code against, but more importantly makes sure that you understand what you are trying to code. Let's make sure you understand how to compute the perceptron learning rule, but doing a simple case by hand.

   Beginning with an input of 0.3 0.7

   And an initial set of weights of -0.6 0.8

   and a class of 1.

   Compute the value of the new weight vector with pen and paper.

   Next steps. Consider this as the data matrix where the first two elements in a row are the two inputs and the third element in a row is the "class" of the output.

   $$\begin{bmatrix} 0.3 & 0.7 & 1.0 \\ -0.5 & 0.3 & -1.0 \\ 0.7 & 0.3 & 1.0 \\ -0.2 & -0.8 & q - 1.0 \end{bmatrix}$$

   using the starting weight -0.6 0.8 try a few cycles through the data by hand and then move on to writing the perceptron rule into code so that you can

automate the updating step. Racket code for some of these functions can be found here.

(a) More Abstract Thinking Each time through the perceptron rule new weights are computed. Imagine these two weights as coordinates for the head of a vector with its base at the origin of a Cartesian plane. Imagine as you iterate that you are rotating your weight vector around an origin.

The decision plane for your network is perpendicular to the vectors and is anchored at the bottom of the arrow. If you could compare this plane to the location of the data points you would see that the rule is learning to find the decision plane that puts all of one class on one side of the line and all of the other class on the other side. That is why it is limited to problems that are linearly separable.

i. Bias is a Good Thing (at least here it is)
These data were selected such that the base of the vector could separate them while anchored at zero. However, for many data sets you not only need to learn what direction to point the vector, but you also need to learn where to anchor the vector. This is done by including a @bold{bias weight}. Add an extra dimension to your weight vector and your inputs. For the inputs it will just be a constant value of 1.0, but this extra, bias weight, will also be learned and allows you to achieve, effectively, a translation away from the origin to be able to separate points that are more heterogeneously scattered.

ii. Geometrical Thinking
- What is the relation between the inner product of two vectors and the cosine of the angle between them?
- What is the **sign** for the cosine of angles less than 90 degrees and those greater than 90 degrees?
- How do these facts help us to answer the question above?
- Why does this reinforce the advice to think *geometrically* when thinking about networks and weight vectors?

3.5.8 The Delta Rule - Homework

The **Delta Rule** is another simple learning rule that is a minimal variation on the perceptron rule. It is used more frequently, and it has the spirit of Hebbian learning, which we will learn more about soon. The homework asks you to write code to test and train an artificial neuron using the delta learning rule.

- For an easy start create some pseudo random linearly separable points on a a sheet of paper. Label one population as **1** and the other population as **-1**.

- For a more challenging set-up create the data programmatically using random numbers and some method that allows you to vary how close or distant the points are to the line of separation, and how many points there are to train on.
- The Delta Learning rule is: $\Delta\ w_i = x_i\ \eta(desired - observed)$
- Submit your code that has your test data in it. Start with an initial random weight and use the delta rule to learn the correct weighting to solve all your training examples. Then test on a new set of points that you did not test on but that are classified according to the same rule. Your code should assess how well the trained rule classifies the test data.

3.5.9 Not all Networks are the Same: Hopfield

- Feedforward
- Recurrent
- Convolutional
- Multilevel
- Supervised
- Unsupervised

The Hopfield network[4] has taught many lessons, both practical and conceptual. Hopfield showed physicists a new realm for their skills and added recurrent (i.e. feedback) connections to network design (output becomes input). He changed the focus from network architecture to that of a dynamical system. Hopfield showed that the network could remember and it could do some error correction, it could reconstruct the "right" answer from faulty input.

1. How a Hopfield Network Operates
   - Each node has a value.
   - Each of those arrowheads has an associated weight.
   - The line with the "x" indicates that there are no self connections.
   - All other connections for all other units are present and go in both directions, and are typically the same in both directions.

Test Your Understanding

- What does the input for a network like this with four nodes look like when framed in the linear algebra terms we just learned?

[4] J J Hopfield, "Neural Networks and Physical Systems with Emergent Collective Computational Abilities.," *Proceedings of the National Academy of Sciences* 79, no. 8 (April 1982): 2554–58, https://doi.org/10.1073/pnas.79.8.2554.

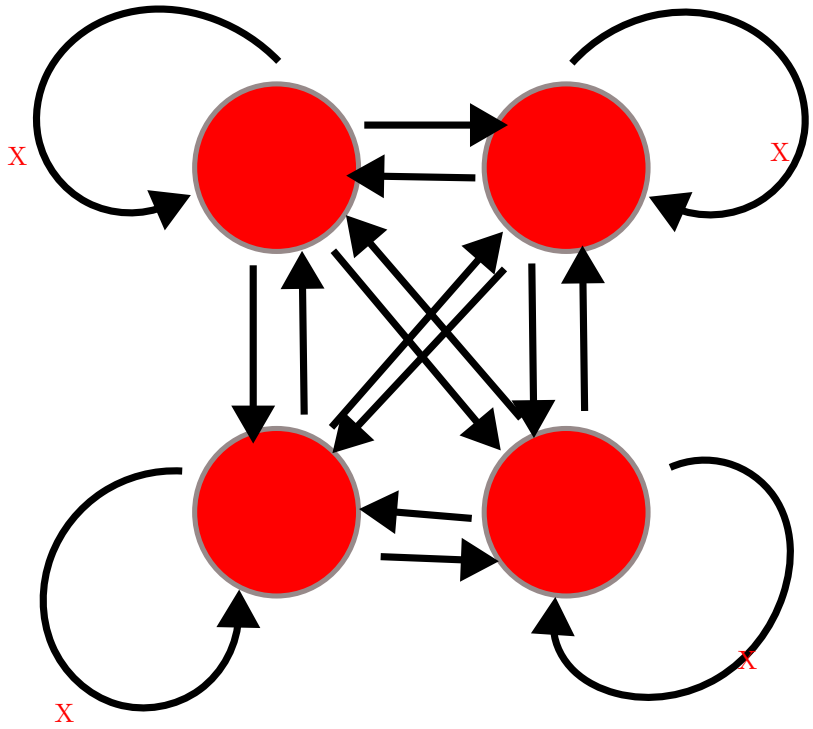


Figure 3.6: Illustration of a Hopfield Network's Connection Pattern With Four Nodes

- A weight is a number associated to each connection. Tell me what the weights should look like in terms of the linear algebra constructs.
- How might we conceive of "running" the network for one cycle in terms of the above?

2. A Worked Example

Inputs can be thought of as vectors. Although I have drawn the network like a square that shape is really independent of the structure of data flow. Each node needs an input and each node will need a weighted contact to all the other nodes. Consider the following two input patterns and the following weight matrix.

$\overrightarrow{a} = [1, 0, 1, 0]^T$

$\overrightarrow{b} = [0, 1, 0, 1]^T$

$$weights = \begin{bmatrix} 0 & -3 & 3 & -3 \\ -3 & 0 & -3 & 3 \\ 3 & -3 & 0 & -3 \\ -3 & 3 & -3 & 0 \end{bmatrix}$$

How do you compute the output? Which comes first: the matrix or the input vector and why?

Hopfield networks use a threshold rule. This non-linearity is, at least metaphorically, like the threshold that says whether a neuron in the brain or in our integrate and fire model fires. For the Hopfield network our threshold rule says:

$$output(t) = \begin{array}{l} 1 \text{ if } t \geq \Theta \\ 0 \text{ if } t < \Theta \end{array}$$

Θ represents the value of our threshold and for now set it to zero.

In Class Activity

- Calculate the output to each of the two input patterns.
- Then to test your intuition, you should guess what output you would get for an input of $[1, 0, 0, 0]^T$.
- Calculate it.
- Is A or B is **closer** to this test input?
- What does it means, in this context, for one vector to be "closer" to another?

(a) An Aside Distance Metrics

Metrics relate to measurement. For some operation to be a distance metric it should meet three intuitive requirements and one that is maybe not as obvious. To measure the distance between two things

we need an operation that is binary. That is, it takes two inputs. In this case that would be our two vectors. It's result should always be **non-negative**. A negative distance would clearly be meaningless. Our output should be **symmetric**. Meaning that $d(A, B) = d(B, A)$. The distance from Waterloo to Toronto ought to come out as the same as going from Toronto to Waterloo. Our metric should be **reflexive**. The distance from anything to itself ought to be zero. Lastly, to be a distance metric, our operation must obey the triangle inequality.

The familiar distance measure of everyday life is the Euclidean. A distance measure calculated as the number of mismatched bits is the *Hamming* distance.

For perceptrons we talked about how the weight vector moved the direction it pointed. Here we don't have the weight vector moving, but you can visualize what is happening as updating a point in space. When we first input our four element vector we have a location in 4-D space. We multiply the first row of our weight matrix against our column of the input vector and we see, in effect, what is the effect on our first element (node) of all the other weighted inputs coming in to it. We then "update" that location. Maybe we flip it from a 1 to a zero (or vice versa). Then we try the next row of the weight matrix to see what happens to the second element. As we change the values of our nodes we are creating new points. The sequence of points is a trajectory that we are tracing in the input space. In this simple situation here we only require one pass to reach the final location, but in other settings we might not. In that case we just keep repeating the process until we do. One of the wonderful insights that Hopfield had was that by conceptualizing this process as an "energy" he could mathematically prove that the process would always reach a resting place.

3. Hebb's Outer Product Rule

Ask yourself

Why is this learning rule called "Hebb's"? Who was the Hebb it is named after?

The strength of a change in a connection is proportionate to the product of the input and outputs, i.e. $\Delta A[i, j] = \eta f[j]g[i]$ and $g[i] = \sum_j \ A[i, j] \ f[j]$ therefore, $\vec{g} = \mathbf{Af}$.[5]

You will want to know

[5]Does it matter that the $\mathbf{W}$ comes first?

- What is an outer product?
- How do you compute it in your programming language of choice?

3.5.10 Hopfield Homework Description: Robustness to Noise (this might take awhile).

Overview of the steps to take:

- Create a small set of random data for input patterns.
- Generate the weights necessary to properly decode the inputs.
- Conceive of a way to randomly corrupt the inputs. Perhaps by flipping some bits and show that your network does correctly decode the uncorrupted inputs.
- Report the accuracy of the output. Explore how the length of the input vector and the number of bits your "flip" impact performance.

More detailed instructions:

- Make the input patterns 2-d, square and of size "n".
- Use a bipolar system and have, roughly, equal numbers of +1s and -1s in your patterns.
- Make a few of them and store them in some sort of data structure.
- Using those patterns, compute the weight matrix with the following equation: $w_{ij} = \frac{1}{N} \sum_{\mu} value_i^{\mu} \times value_j^{\mu}$ Where N is the size of the patterns, that is how many "neurons". μ is an index for each of the patterns, and i and j refer to the neurons in the pattern.
- Program an **asynchronous** updating rule, run your network until it stabilizes, and then show that you get back what you put in.
- Then do the same for at least one disrupted pattern (where you flipped a couple of bits around.)

Chapter 4

Dynamics

Why are **differential equations** an important technique for computational modelling in psychology and neuroscience?

Spiking neuron models rely on differential equation to capture the evolution of voltage as a function of time. Dynamics is the study of things changing over time, and neural dynamics gives us important insights into brain and neuronal activity normally and as a consequence of disease.

An interest in dynamical systems has been a part of computational neuroscience since the days of Hodgkin and Huxley (more on them later). But with the development of cheaper and more powerful computing capacities their use as expanded practically and theoretically. It is increasingly feasible to simulate complex models in high dimensions. This advance parallels the use of multiple electrode recordings that yield high dimensional data.

To ground our understanding with the use of differential equations in computational neuroscience we will produce simple, single neuron simulations of the integrate and fire variety. An excellent and concise summary of these ideas, one that I drew upon heavily for this course is.[1]

4.1 Beginning to Think Dynamically

Assume you have a derivative that is equal to $x - x^3$ (see Figure 4.1). What would it mean to "solve" this equation?

What you want is some *function* of the variable x that will equal what you get when you take its derivative. For this function that is easy enough. You simply integrate. But what if we think of x as dynamic. It is not fixed. It varies with time, like the voltage inside a neuron, the air temperature, or the strength

[1] David Terman, "An Introduction to Dynamical Systems and Neuronal Dynamics," in *Tutorials in Mathematical Biosciences I* (Springer Berlin Heidelberg, 2005), 21–68, `https://doi.org/10.1007/978-3-540-31544-5_2`.

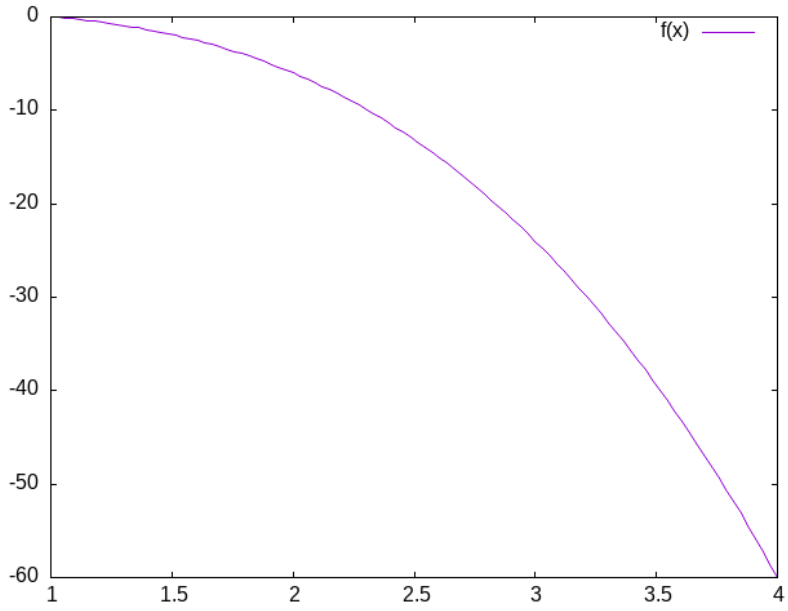


Figure 4.1: A plot of $f(x) = x - x^3$

of a memory trace. Then your equation becomes, $\frac{dx}{dt} = x - x^3$. Integrate that.[2] In this case we can find an analytical[3] solution. But we don't have to do much to our equations to make such solutions very difficult or impossible. For most of the use practical use cases in computational neuroscience we do not solve these equations analytically. Rather, we use the computer to approximate their behavior. Most of the equations in neuroscience, and biology generally for that matter, are well behaved enough that we can get away with this.

We will now review the action potential and lay the ground work for writing our own code to simulate a neuron's firing.

4.2 The Action Potential

How is an action potential like a flush toilet?

4.3 A Mathematical Aside

I often find that psychology students let mathematical notation get in their way of understanding. The unfamiliar symbols lead people to skip the most

[2] It is not *that* hard, and Claude.AI (as of *¡2024-12-19 Thu¿*) can do it for you. But it is substantially harder than before.

[3] What does "analytical" mean here?

important part of computational papers. The most important thing to learn is that mathematical notation is just like an emoji: a concise way to express a more complex idea. Once you learn the "slang" you can look beyond the characters. Here is an example of how simple math can start to look forbidding when notated.

What does $\sum_{\forall x \in \{1,2,3\}} x = 6$ really mean?

Another confusing feature of mathematical notation is that there are often many ways to say the same thing and different mathematical communities have different conventions. But just because you cannot speak French you do not assume the French are really saying something different. So, when faced with a strange mathematical expression be open to the idea that it is just another way to say something you are already familiar with.

For example, $\frac{dx}{dt}$ $\dot{x}$ x' $f'(x)$ are all different ways of talking about derivatives and derivatives are at the heart of differential equations.

4.4 Derivatives are Slopes

1. What is a slope?
2. When in doubt return to definition.
3. Deriving the definition of a derivative.
4. What is the definition of a derivative?

4.4.1 Your computer is a tool for exploration

4.4.2 To approximate the derivative of a curve at a point draw a straight line close by

You pick two points that are "close enough" and you get an answer that is "close enough." If your answer isn't "close enough" then you move your points closer, until *in the limit* there is an infinitesimal distance between them.

Definition of a Derivative

$$\frac{dx}{dy} = \frac{f(x+\Delta x)-f(x)}{x+\Delta x-x}$$

4.5 Digression:

To write math for publication you will probably need to know a little bit of LaTeX. LaTeX versions can be found for all sorts of environments and operating systems. To get started try writing LaTeX in VS Code. A guide for word processor users can be found here.

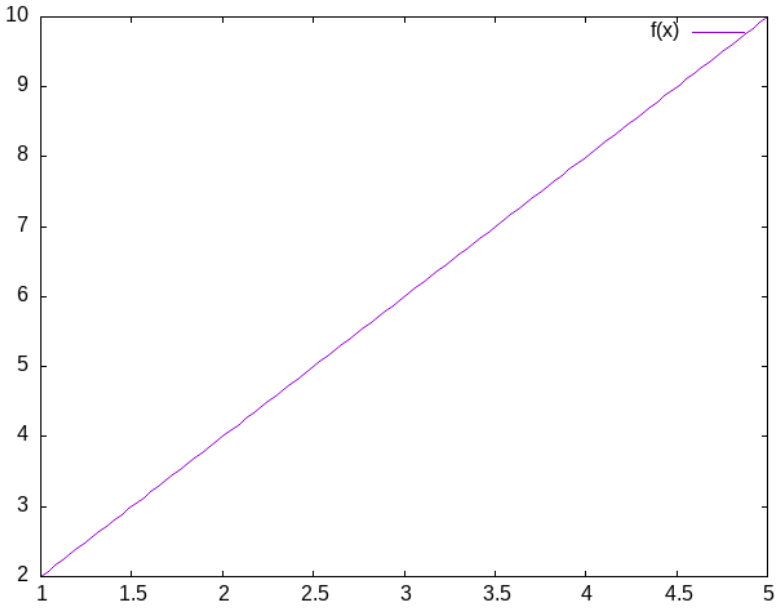


Figure 4.2: What is the slope of this line?

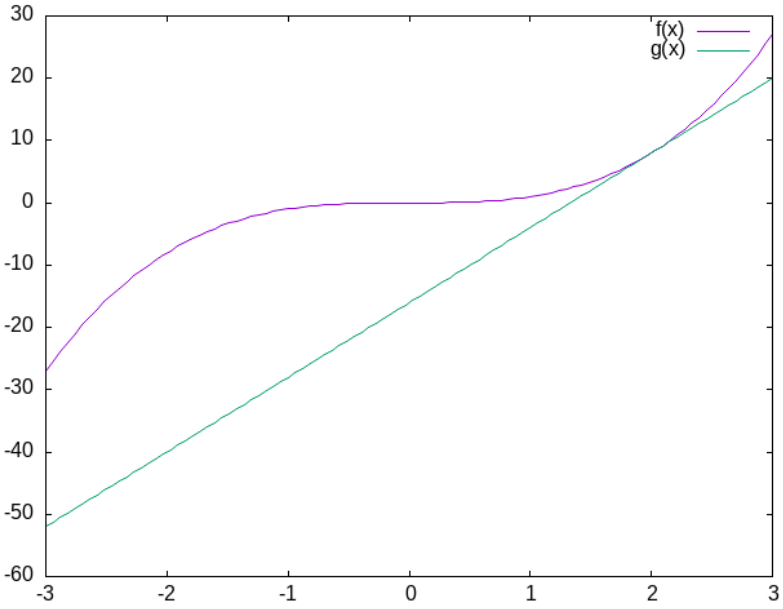


Figure 4.3: What is the slope of this curve? And what is missing from that question that you need to know to give an answer?

4.5.1 Using Derivatives to Solve Problems With a Computer

What is a square root? You might want to review our earlier exercise to see how the derivative is playing a role in our solution.

4.6 Practice Simulating With DEs

4.6.1 Frictionless Springs

We want to code neurons, but to get there we should feel comfortable with the underlying tool or we won't be able to adapt it or re-use it for some new purpose. We will start with a very simple and concrete example: the frictionless spring.

$\frac{d^2s}{dt^2} = -P\ s$

where 's' refers to space, 't' refers to time, and 'P' is a constant, often called the spring constant, that indicates how stiff or springy the spring is.

Imagine that you knew this derivative. It would tell you how much space the spring head would move for a given, very small, increment of time. We could then just add this to our current position to get the new position and repeat. This method of using a derivative to iterate forward is sometimes called the Euler method.

Returning to our definition of the derivative:

$\frac{s(t+\Delta t)-s(t)}{\Delta t} = velocity \approx \frac{ds}{dt}$

But our spring equation is not given in terms of the velocity it is given in terms of the acceleration which is the second derivative. Therefore, to find our new position we need the velocity, but we only have the acceleration. However, if we knew the acceleration and the velocity we could use that to calculate the new velocity. Unfortunately we don't know the velocity, unless ... , maybe we could just assume something. Let's say it is zero because we have started our process where we have stretched the spring, and are holding it, just before letting it go. We have asserted an *initial value*.

How will our velocity change with time?

$\frac{v(t+\Delta t)-v(t)}{\Delta t} = acceleration \approx \frac{dv}{dt} = \frac{d^2s}{dt^2}$

And we have a formula for this. We can now bootstrap our simulation.

Note the similarity of the two functions. You could write a helper function that was generic to this pattern of *old value + rate of change times the time step*, and just used the pertinent values.

How do we know the formula for acceleration? We were given it!

```
import localcode.spring as s
s.plot_spring()
```

1. Now You Try **Damped Oscillators**

   Provide the code and a plot for the damped oscillator. It has the formula of

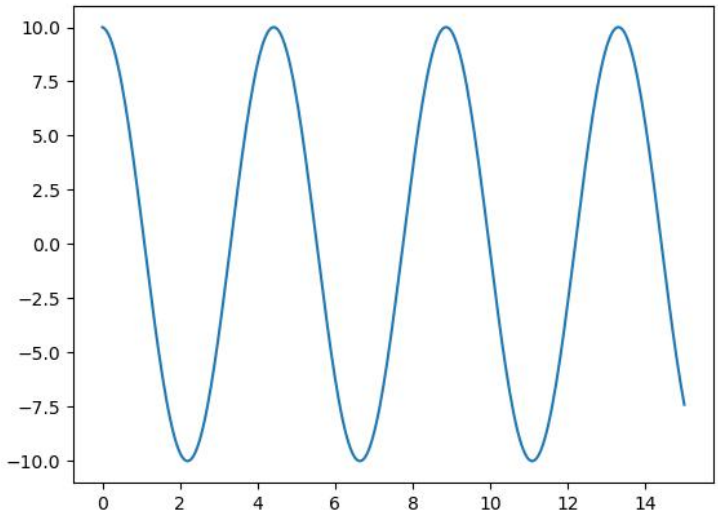


Figure 4.4: Spring constant of 2 with a starting position of 10.

$\frac{d^2s}{dt^2} = -P\ s(t) - k\ v(t)$